

Aufgaben zu Polymorphie in Python

Aufgabe 1: Polymorphie durch Vererbung (Fahrzeugverwaltung)

Sie entwickeln ein Verwaltungssystem für unterschiedliche Fahrzeugtypen (z.B. Auto, Fahrrad, Boot). Alle Fahrzeuge sollen eine Methode „bewegen()“ besitzen, die ihr jeweiliges Bewegungsverhalten beschreibt.

- a) Implementieren Sie eine Basisklasse „Fahrzeug“ mit einer abstrakten Methode „bewegen()“.
- b) Leiten Sie mindestens drei konkrete Fahrzeugklassen ab und implementieren Sie jeweils ein spezifisches Verhalten.
- c) Schreiben Sie eine Funktion, die eine Liste von Fahrzeugen entgegennimmt und polymorph die Methode „bewegen()“ aufruft.
- d) Testen Sie Ihre Implementierung mit verschiedenen Objekten.

Aufgabe 2: Polymorphie durch Duck Typing (Benachrichtigungssystem)

Sie entwickeln ein Benachrichtigungssystem, das Nachrichten über verschiedene Kanäle versendet (z.B. E-Mail, SMS, Push-Nachricht).

- a) Implementieren Sie drei Klassen mit jeweils einer Methode „senden(nachricht)“.
- b) Entwickeln Sie eine Funktion, die ein beliebiges Objekt akzeptiert und die Methode „senden()“ aufruft, ohne eine gemeinsame Basisklasse zu verwenden.
- c) Erläutern Sie, weshalb es sich hierbei um Polymorphie handelt.
- d) Ergänzen Sie einen weiteren Kanal, ohne die bestehende Versandfunktion zu verändern.

Aufgabe 3: Polymorphie mit gemeinsamen Schnittstellen (Gehaltsberechnung)

In einem Unternehmen gibt es unterschiedliche Mitarbeitertypen (Festangestellte, Freelancer, Praktikanten), deren Gehalt jeweils unterschiedlich berechnet wird.

- a) Definieren Sie eine gemeinsame Schnittstelle oder abstrakte Basisklasse mit der Methode „berechne_gehalt()“.
- b) Implementieren Sie mindestens drei konkrete Klassen mit jeweils eigener Berechnungslogik.
- c) Schreiben Sie eine Auswertungsfunktion, die für eine Liste von Mitarbeitenden das Gesamtgehalt berechnet.
- d) Reflektieren Sie, welche Vorteile die polymorphe Struktur gegenüber einer if-elif-Konstruktion bietet.

Aufgabe 4: Polymorphie in einem spielerischen Kontext (Tiergeräusche im Zoo)

Sie entwickeln eine kleine Anwendung für einen digitalen Zoo. Unterschiedliche Tiere sollen Geräusche von sich geben.

- a) Implementieren Sie eine Basisklasse „Tier“ mit einer Methode „make_geraeusch()“.
- b) Leiten Sie mehrere Tierklassen (z.B. Hund, Katze, Papagei) ab und überschreiben Sie die Methode entsprechend.
- c) Erstellen Sie eine Funktion „zoo_show(tiere)“, die für alle übergebenen Tiere das jeweilige Geräusch ausgibt.
- d) Erweitern Sie das System um ein weiteres Tier, ohne die bestehende Show-Funktion zu verändern.